Project Progress Report: Diffusion Text Generation

Prepared on April 10, 2026

The project started with a simple question: can text generation work with a diffusion-style process instead of the usual left-to-right setup? The original goal was exploratory. We wanted to see whether we could train a discrete diffusion language model end to end, keep training stable, and get checkpoints that produced anything coherent at all.

The first stage of the work happened inside the original `Diffusion_Testing` repository. That repo eventually held several experiments at once: from-scratch diffusion models, benchmark scripts, backup checkpoints, and a separate `DiffuLLM` path for comparing against Qwen baselines. That was useful at first, but over time it became harder to reason about. Different checkpoints expected different model definitions, some scripts only matched older architectures, and even simple evaluation started taking too much effort because every run had to be traced back to the exact code that produced it.

A lot of the recent work in that original repo was really about cleaning up our understanding of what we already had. We went through old checkpoints, loaded them one by one, and tested them with simple prompts, especially short recipe prompts like asking for a cake recipe. That gave us a practical baseline. Most of the checkpoints were technically runnable, but the generations were not strong. Some outputs repeated punctuation, some repeated a few common words, and some looked like the model was almost following the prompt before falling apart.

That is what pushed the project into a second phase. Instead of continuing to pile fixes and new training attempts into the original folder, we created `Diffusion_Testing2` as a cleaner place to work. This was not just a rename. The newer folder represented a shift in approach: instead of relying only on the earlier from-scratch setups, it used a pretrained Qwen backbone with LoRA adapters and a masked diffusion decoding process. The idea was that starting from a stronger language model might give the diffusion objective a better foundation.

Once `Diffusion_Testing2` became the active repo, the next challenge was understanding how everything in it actually worked. We traced the training flow, checkpoint format, and generation path. One thing we ran into fairly quickly was that prompt-based evaluation was not as straightforward as we needed. For this project, it is not enough for the model to produce random text; we want to know whether it can respond to a specific request. So part of the work here was making sure we could test checkpoints in a way that preserved the prompt and only generated the completion.

After that, the biggest technical effort was getting the larger Qwen-based training run working on Rutgers iLab. This involved more infrastructure debugging than model debugging at first. The SLURM jobs had to be adjusted, the environment setup had to be fixed, and the existing virtual environment turned out not to be reliable enough for the run. We solved that by updating the scripts to use the working conda environment instead. Once those issues were fixed, the training job finally became stable and we were able to leave it running.

That iLab run is the clearest sign of progress so far. It has produced a real sequence of checkpoints, from `step_500` through `step_6500`, and the job is still running. To make evaluation manageable, we also created a smaller inference-only checkpoint from the latest saved state and downloaded it locally. We then ran the same recipe-style prompts locally against the newer checkpoint and compared the output to what we had seen from the older models.

The honest result is mixed. On the positive side, the pipeline now works much better than before. We have a cleaner repo structure, a Qwen-based diffusion model that trains on iLab, checkpoints that can be pulled down and tested locally, and a much clearer sense of which part of the stack is doing what. On the negative side, the actual text quality is still not where we want it to be. The latest checkpoint shows slightly more awareness of prompt structure than the earlier ones, but it still breaks into repeated fillers and unstable token patterns instead of producing a clean recipe.

Current snapshot

- The project began as an experiment in diffusion-based text generation inside the original `Diffusion_Testing` repo.
- That repo became crowded with multiple model paths and checkpoint formats, which made evaluation harder over time.
- `Diffusion_Testing2` was created as a cleaner Qwen-based training and inference workspace.
- The Rutgers iLab pipeline is now running reliably and has reached at least `step_6500`.
- The latest checkpoint can be tested locally, but generation quality is still the main unsolved problem.